

IP Architecture

Algorithms for Internet Protocol Modules

1. Header Adding Module

1. Module Definition & Architectural Context

The Header Adding Module operates at the Network Layer (Layer 3) of the OSI model. Its primary function is to receive a data payload from the Transport Layer (e.g., TCP or UDP) and encapsulate it into an Internet Protocol (IP) datagram by prepending a standard IP header. This enables process-to-process delivery to be routed across the network to a specific destination host.

2. Inputs, Outputs, and Data Structures

- **Input Queue (Q_in):** Contains the data payload (segment/datagram) from the upper Transport Layer, alongside metadata (Destination IP, Protocol ID).
- **Output Queue (Q_out):** Buffers the encapsulated IP packets before passing them to the Processing Module (Routing Algorithm).
- **Header Data Structure:** A contiguous memory block of 5, 32-bit words (160 bits total, equating to a minimum 20-byte header) representing the standard IPv4 header.

3. IP Header Encapsulation Algorithm

Algorithm: Encapsulate_IP_Header

```

1.  Wait until Q_in is not empty.
2.  Dequeue (Payload, Dest_IP, Protocol_ID) from Q_in.
3.  Let H be a 20-byte IP Header structure initialized to 0.

    // Step 3.1: Assign Static & Metadata Fields
4.  H.Version = 4                      // 4 bits: IPv4
5.  H.IHL = 5                          // 4 bits: Header length in 32-
    bit words (5 * 4 = 20 bytes)
6.  H.TOS = 0                          // 8 bits: Type of Service (
    default routine)
7.  H.Total_Length = 20 + len(Payload) // 16 bits: Header + Payload
    length in bytes
8.  H.Identification = generate_ID()   // 16 bits: Unique sequence
    number for fragmentation
9.  H.Flags = 000                      // 3 bits: Reserved, Don't
    Fragment, More Fragments
10. H.Fragment_Offset = 0              // 13 bits: 0 for an initial
    unfragmented packet
11. H.TTL = 64                        // 8 bits: Time to Live to
    prevent routing loops
12. H.Protocol = Protocol_ID           // 8 bits: Protocol
    encapsulated (e.g., 6 for TCP, 17 for UDP)
13. H.Source_IP = Local_Host_IP       // 32 bits: Sender's IP address
14. H.Dest_IP = Dest_IP               // 32 bits: Receiver's IP
    address

```

```

    // Step 3.2: Header Checksum Computation
15. H.Checksum = 0           // 16 bits: Temporarily set to
    zero for calculation
16. H.Checksum = Calculate_Checksum(H)

    // Step 3.3: Encapsulation & Forwarding
17. IP_Packet = Concatenate(H, Payload)
18. Enqueue IP_Packet into Q_out (for the Processing Module)
19. Return to Step 1.

```

4. Mathematical Computation: Header Checksum

To ensure header integrity during transmission, the checksum is calculated strictly over the 20-byte header, omitting the payload data.

1. Divide the 20-byte header into ten 16-bit words: $W_1, W_2, \dots, W_{10}$.
2. Compute the sum using one's complement arithmetic:

$$Sum = \sum_{i=1}^{10} W_i$$

3. If a carry-out occurs from the most significant bit during addition, add it back to the least significant bit.
4. The final Checksum is the one's complement (bitwise NOT) of the sum:

$$Checksum = \sim Sum$$

(Note: Upper-layer protocols, such as TCP or UDP, are responsible for verifying their own payload data integrity.)

5. Complexity Analysis

- **Time Complexity:** $\mathcal{O}(1)$. The header size is fixed at 20 bytes (assuming no options), meaning the checksum calculation and field assignments execute in constant time regardless of the payload's size.
- **Space Complexity:** $\mathcal{O}(1)$ auxiliary space per packet, as the memory allocation for the standard IP header is strictly bounded to 20 bytes.

2. Processing Module (Routing Algorithm)

1. Module Definition & Architectural Context

The Processing Module sits at the core of the Network Layer. It receives IP packets from two sources — the Header Adding Module (locally generated traffic) and the Data Link Layer (transit traffic arriving from other hosts/routers). Its job is to validate the header, decide the next hop using a routing table lookup, and pass the packet onward to the Forwarding Module.

2. Inputs, Outputs, and Data Structures

- **Input Queues:** IP packets from Header Adding Module; IP packets from Data Link Layer (incoming interfaces).
- **Output:** (IP_Packet, Next_Hop, Outgoing_Interface) tuple sent to the Forwarding Module.
- **Routing_Table:** Set of entries {Network_Prefix, Subnet_Mask, Next_Hop_IP, Interface}, typically implemented as a trie (Patricia trie) for longest-prefix matching.

3. Routing Algorithm

Algorithm: Process_IP_Packet

```

1.  Wait until an input queue (local or link-layer) is not empty.
2.  Dequeue IP_Packet; let H = IP_Packet.Header.
3.  Verify H.Checksum against a recomputed checksum.
4.      If mismatch: discard packet, return to Step 1.
5.  If H.Dest_IP == Local_Host_IP:
6.      Strip header, deliver Payload to upper (Transport) layer via
    Q_out.
7.      Return to Step 1.
8.  H.TTL = H.TTL - 1
9.  If H.TTL == 0:
10.     Generate ICMP "Time Exceeded", discard packet, return to Step
    1.
11. (Next_Hop, Interface) = Longest_Prefix_Match(Routing_Table, H.
    Dest_IP)
12. If no match found:
13.     Generate ICMP "Destination Unreachable", discard packet, return
    to Step 1.
14. H.Checksum = 0; H.Checksum = Calculate_Checksum(H)    // recompute
    after TTL change
15. Enqueue (IP_Packet, Next_Hop, Interface) to Forwarding Module.
16. Return to Step 1.
```

4. Mathematical Basis: Longest Prefix Match

For a destination address D and routing entries (P_i, M_i) where P_i is a prefix and M_i its mask length:

$$\text{Match} = \arg \max_i \{M_i \mid (D \& \text{Mask}(M_i)) = P_i\}$$

The entry with the greatest mask length M_i among all matching prefixes is selected.

5. Complexity Analysis

- **Time Complexity:** $\mathcal{O}(W)$ using a binary trie, where $W = 32$ (IPv4 address width) — independent of the number of routing entries n . A naive linear scan is $\mathcal{O}(n)$.
- **Space Complexity:** $\mathcal{O}(n)$ for storing n routing table entries.

3. Forwarding Module

1. Module Definition & Architectural Context

The Forwarding Module takes the routing decision made by the Processing Module and physically prepares the packet for transmission on the correct outgoing interface, consulting the Routing Table for interface-level details (e.g., link-layer resolution) before handing off to the Fragmentation Module.

2. Inputs, Outputs, and Data Structures

- **Input:** (IP_Packet, Next_Hop, Interface) from the Processing Module.
- **Output:** (IP_Packet, Next_Hop, Interface) enqueued toward the Fragmentation Module.
- **Routing_Table:** Consulted to confirm interface state (up/down) and any interface-specific parameters.

3. Forwarding Algorithm

Algorithm: Forward_IP_Packet

```
1.  Wait until Forwarding input queue is not empty.
2.  Dequeue (IP_Packet, Next_Hop, Interface).
3.  If Interface.Status != UP:
4.      Generate ICMP "Destination Unreachable", discard packet, return
      to Step 1.
5.  If Next_Hop is on a directly connected network:
6.      Effective_Next_Hop = IP_Packet.Header.Dest_IP
7.  Else:
8.      Effective_Next_Hop = Next_Hop          // gateway address
9.  Enqueue (IP_Packet, Effective_Next_Hop, Interface) to Fragmentation
      Module.
10. Return to Step 1.
```

4. Complexity Analysis

- **Time Complexity:** $\mathcal{O}(1)$ — interface status check and next-hop resolution are constant-time table lookups once the route is already known.
- **Space Complexity:** $\mathcal{O}(1)$ auxiliary space per packet.

4. Fragmentation Module

1. Module Definition & Architectural Context

The Fragmentation Module ensures an outgoing IP packet does not exceed the Maximum Transmission Unit (MTU) of the chosen outgoing interface. If it does, the module splits the packet into multiple smaller IP fragments, each independently routable and reassembled at the destination.

2. Inputs, Outputs, and Data Structures

- **Input:** (IP_Packet, Next_Hop, Interface) from the Forwarding Module.
- **MTU_Table:** {Interface → MTU_Value} lookup table.
- **Output:** One or more (Fragment, Next_Hop) pairs enqueued to the Data Link Layer.

3. Fragmentation Algorithm

```

Algorithm: Fragment_IP_Packet

1.  Wait until Fragmentation input queue is not empty.
2.  Dequeue (IP_Packet, Next_Hop, Interface); H = IP_Packet.Header.
3.  MTU = MTU_Table[Interface]
4.  If H.Total_Length <= MTU:
5.      Enqueue (IP_Packet, Next_Hop) to Data Link Layer output queue.
6.      Return to Step 1.
7.  If H.Flags.Dont_Fragment == 1:
8.      Generate ICMP "Fragmentation Needed", discard packet, return to
        Step 1.
9.  Data_Per_Fragment = floor((MTU - 20) / 8) * 8          // must be
        multiple of 8 bytes
10. Offset = 0
11. Remaining_Data = Payload
12. While Remaining_Data is not empty:
13.     Chunk = first min(Data_Per_Fragment, len(Remaining_Data)) bytes
        of Remaining_Data
14.     Create Fragment_Header = copy of H
15.     Fragment_Header.Total_Length = 20 + len(Chunk)
16.     Fragment_Header.Fragment_Offset = Offset / 8
17.     Fragment_Header.Flags.More_Fragments = (Remaining_Data has more
        bytes after Chunk) ? 1 : 0
18.     Fragment_Header.Checksum = 0
19.     Fragment_Header.Checksum = Calculate_Checksum(Fragment_Header)
20.     Fragment = Concatenate(Fragment_Header, Chunk)
21.     Enqueue (Fragment, Next_Hop) to Data Link Layer output queue.
22.     Offset = Offset + len(Chunk)
23.     Remove Chunk from front of Remaining_Data.
24. Return to Step 1.

```

4. Mathematical Constraint

Number of fragments required for payload of size L bytes over an MTU M :

$$N = \left\lceil \frac{L}{\lfloor (M - 20)/8 \rfloor \times 8} \right\rceil$$

Each fragment's offset field stores the offset in 8-byte units, so $\text{Fragment_Offset} = \frac{\text{Offset}}{8}$.

5. Complexity Analysis

- **Time Complexity:** $\mathcal{O}(N)$, where N is the number of fragments produced — each requiring $\mathcal{O}(1)$ header construction and checksum computation.
- **Space Complexity:** $\mathcal{O}(N)$ for buffering the N outgoing fragments.

5. Reassembly Module

1. Module Definition & Architectural Context

The Reassembly Module runs at the receiving host. It collects IP fragments arriving (possibly out of order, from possibly different paths) from the Data Link Layer, groups them by originating datagram, and reconstructs the original payload before delivering it to the upper layer.

2. Inputs, Outputs, and Data Structures

- **Input:** IP fragments from the Data Link Layer.
- **Reassembly_Table:** Keyed by (Source_IP, Dest_IP, Protocol, Identification) → {Fragment_Buffer[], Bytes_Received, Total_Length, Timer}.
- **Output:** Reassembled Payload delivered to the upper (Transport) layer.

3. Reassembly Algorithm

Algorithm: Reassemble_IP_Fragment

```

1.  Wait until Reassembly input queue is not empty.
2.  Dequeue Fragment; H = Fragment.Header.
3.  Key = (H.Source_IP, H.Dest_IP, H.Protocol, H.Identification)
4.  If Key not in Reassembly_Table:
5.      Create new entry: Reassembly_Table[Key] = {Buffer: empty,
                                                    Bytes_Received: 0,
                                                    Total_Length:
                                                    UNKNOWN, Timer: start(60s)}
6.  Entry = Reassembly_Table[Key]
7.  Insert Fragment's data into Entry.Buffer at byte offset (H.
    Fragment_Offset * 8).
8.  Entry.Bytes_Received += len(Fragment.Payload)
9.  If H.Flags.More_Fragments == 0:
10.     Entry.Total_Length = (H.Fragment_Offset * 8) + len(Fragment.
        Payload)
11. If Entry.Total_Length != UNKNOWN AND Entry.Bytes_Received == Entry.
    Total_Length
    AND Buffer has no gaps:
12.     Complete_Payload = Entry.Buffer
13.     Deliver Complete_Payload to upper layer.
14.     Cancel Entry.Timer; Delete Reassembly_Table[Key].
15. Return to Step 1.
```

// Concurrent process - runs per Reassembly_Table entry

Algorithm: Reassembly_Timeout_Handler(Key)

```

1.  On Timer expiry (60s elapsed, e.g., per RFC 791 default):
2.      Generate ICMP "Time Exceeded - Fragment Reassembly Time
    Exceeded".
3.      Discard Reassembly_Table[Key].Buffer.
4.      Delete Reassembly_Table[Key].
```

4. Mathematical Completion Condition

Let fragments have offsets $O_1 < O_2 < \dots < O_k$ (in bytes) and lengths $L_1, \dots, L_k$. Reassembly is complete when the fragments form a contiguous run from 0 to the final length T :

$$O_1 = 0, \quad O_{j+1} = O_j + L_j \quad \forall j, \quad \text{and the last fragment has MF} = 0$$

5. Complexity Analysis

- **Time Complexity:** $\mathcal{O}(1)$ amortized per fragment insertion (hash-table lookup by Key + offset-indexed buffer write); $\mathcal{O}(k)$ to verify contiguity across k fragments of one datagram in the worst case.
- **Space Complexity:** $\mathcal{O}(T)$ per in-progress datagram, where T is the total reassembled datagram length, bounded across all concurrent entries in the `Reassembly_Table`.

6. Summary Table

Module	Core Data Structure	Time Complexity	Space Complexity
Header Adding	20-byte header buffer	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Processing (Routing)	Routing Table (trie)	$\mathcal{O}(W) \approx \mathcal{O}(32)$	$\mathcal{O}(n)$ entries
Forwarding	Routing/Interface Table	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Fragmentation	MTU Table	$\mathcal{O}(N)$ fragments	$\mathcal{O}(N)$
Reassembly	Reassembly Table (hash)	$\mathcal{O}(1)$ amortized/fragment	$\mathcal{O}(T)$ per datagram

Source Code:

The Python simulation for these IPv4 modules is available on GitHub:

<https://github.com/AryanAI0035/ipv4-architecture-simulator>

7. Conclusion

In conclusion, this assignment comprehensively formulates the five fundamental modules of the Internet Protocol. By mathematically and algorithmically defining header encapsulation, routing, forwarding, fragmentation, and reassembly, we establish a robust understanding of network-layer data transmission mechanics.

References

-
- [1] Postel, J. (1981). *Internet Protocol* (RFC 791). Internet Engineering Task Force.
 - [2] Kurose, J. F., & Ross, K. W. (2021). *Computer Networking: A Top-Down Approach* (8th ed.). Pearson.
 - [3] Stevens, W. R., Fall, K. R. (2011). *TCP/IP Illustrated, Volume 1: The Protocols* (2nd ed.). Addison-Wesley.